

TECHNICAL DESIGN REVIEW · HLD + LLD

Distributed SMS Service

A polyglot, event-driven messaging platform with a transactional outbox — durable acceptance, exactly-once delivery, and an operable failure model.

Java 21 · Spring Boot 3.2

Go

Apache Kafka

Redis

MongoDB

H2

Docker

2

decoupled microservices

exactly-once

to recipient & to storage

5/5

failure modes resolved

Krishna Thakur

11 June 2026 · v1.0

The Problem & The Approach

⚠ The naïve flow — and why it breaks

```
request → vendor call → Kafka publish → 200 OK
      (everything on the request thread)
```

- **Event loss** — if Kafka is down, the publish is lost silently.
- **Service stalls** — under load the producer buffer fills and request threads block.
- **Duplicate SMS** — a retry re-runs the vendor call, messaging the user twice.
- **Coupling** — the sender fails whenever the store is slow or down.

✅ The principle: decouple acceptance from work

```
request → durable outbox write → 202 Accepted
      background worker does the real work later
```

- The request path does only a **fast local DB write** — never blocks on Kafka/vendor.
- A downstream outage just makes rows **queue**, then drain on recovery.
- A two-step state machine + idempotency keys make delivery **exactly-once**.
- Stuck messages are **dead-lettered** and replayable — never lost, never infinite.

GUARANTEE 1

Durable acceptance

202 means it will never be lost

GUARANTEE 2

Exactly-once to recipient

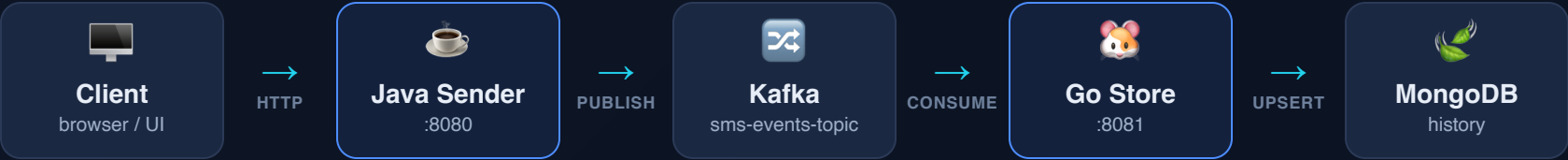
no double-send on retry/crash

GUARANTEE 3

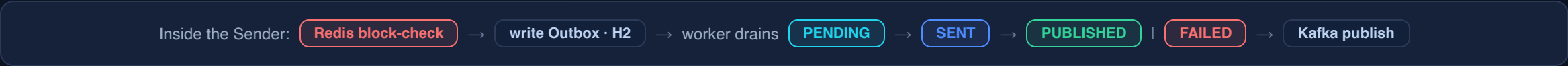
Exactly-once storage

one record per request in Mongo

System Architecture



The two services never call each other directly — Kafka is the only link. History is read back from the Go Store on `:8081`.



Java Sender
Synchronous API. Accepts requests, runs the outbox worker, publishes events. **Stays fast & available.**

Go Store
Async worker. Consumes events, persists history, serves queries. **Processes at its own pace.**

Infra
Redis blocklist · Kafka bus · Mongo store · H2 outbox.

Inside the Java Sender

 Layered responsibilities

COMPONENT	RESPONSIBILITY
SmsController	send · health · block / unblock / check
OutboxController	list outbox · list failed · replay
SmsServiceImpl	Redis check → write outbox row → 202
OutboxProcessor	@Scheduled worker: state machine + backoff
MockSmsVendorService	deterministic vendor, idempotent by key
KafkaSmsEventPublisher	serialize + publish (blocking, worker thread)
ApiKeyAuthFilter	guards the admin surface (X-API-Key)

 Request path (synchronous)

```
POST /v1/sms/send
| SISEMBER blocked_users → blocked? 403
| INSERT outbox (PENDING, id=UUID)
└→ 202 Accepted {messageId} ≈ ms
```

 Worker loop (every 1s)

```
PENDING → vendor.send(id,...) → SENT
SENT → publish(eventId) → PUBLISHED
fail → backoff 2→4→8... → retry
10x → FAILED (dead-letter)
```

The Go Store & the Gateway

 Go packages

PACKAGE	RESPONSIBILITY
kafka	consumer loop: read → unmarshal → save
services	Mongo upsert keyed by eventId
handlers	HTTP: health, message history
models	shared SmsEvent (json+bson)
config	env-driven config + defaults

 Consumer loop (goroutine)

```
for {
  msg = reader.ReadMessage(ctx) // blocks
  json.Unmarshal(msg, &event)
  svc.SaveEvent(event) // upsert
} // ctx cancel → graceful stop
```

 Demo gateway (Go reverse proxy)

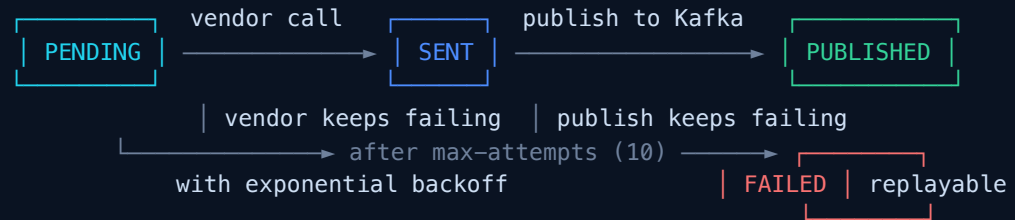
Serves the UI and routes all API traffic so the browser sees **one origin** — CORS disappears.

```
/api/sender/* → :8080 (+ inject X-API-Key)
/api/store/* → :8081
/*           → static UI (:3000)
```

 Why the gateway matters

- One origin → **no CORS**, no backend changes.
- Injects the admin key → secret **never in browser code**.
- Acts as the trusted edge: direct hits to :8080 are rejected.

The Transactional Outbox



Why two steps

Splitting "**call the vendor**" (PENDING→SENT) from "**publish the event**" (SENT→PUBLISHED) is the no-duplicate-SMS guarantee. Once a row is **SENT**, a Kafka failure only retries the *publish* — **the vendor is never called again**.



Backoff & dead-letter

Each failure doubles the delay (`2s→4s→8s` ... capped 5 min) via a future `nextAttemptAt`. The ~13-min window **rides out a transient outage** instead of quarantining good messages; only a genuinely stuck row hits **FAILED**.

End-to-End Flow & Idempotency

● Happy path (send → stored)

```
1 POST /v1/sms/send
2 SISEMEMBER blocked_users → ok
3 INSERT outbox PENDING
4 return 202 {messageId} ≈ ms
5 worker: vendor.send → SENT
6 worker: publish → PUBLISHED
7 Go: consume → upsert(eventId)
8 Mongo: 1 document
9 UI History → visible
```

🔗 The idempotency chain

```
Vendor side: key = outbox row id
              replay → cached result → SMS once
Storage side: eventId = row id
              redelivery → upsert → 1 doc
```

✅ Net guarantee

Exactly-once **to the recipient** and exactly-once **in storage** — even though Kafka in the middle is *at-least-once* by design. The two ends are idempotent, so duplicates in the stream are harmless.

Failure Modes — Found & Fixed

Each was discovered by stress-testing the pipeline (kill Kafka, crash mid-step, redeliver) and closed with a concrete change.

FAILURE MODE	RESOLUTION	STATUS
Event lost when Kafka is down (fire-and-forget on request thread)	Outbox pattern — durable H2 queue + async worker; request path never blocks on Kafka	FIXED
Duplicate SMS on retry / worker crash	Two-step state machine + idempotency key passed to the vendor	FIXED
Duplicate events in MongoDB (at-least-once delivery)	Consumer upsert keyed by <code>eventId</code> + sparse unique index	FIXED
Poison messages retry forever	Exponential backoff + dead-letter (FAILED) + admin replay endpoint	FIXED
Open admin endpoints (anyone could block / replay)	API-key filter on the admin surface + gateway key injection	FIXED

Live-validated: during testing, a Kafka restart produced 3 duplicate Mongo rows — exactly the predicted at-least-once behaviour — which the `eventId` upsert then collapsed back to one. The diagnosis and the fix matched.

Security & Technology Choices

🔒 Admin authentication

Browser (no secret)

↳ Gateway (holds key, injects header)

↳ Java filter → ✓ 200

Attacker → Java filter → ✗ 401

X-API-Key guards /block/** & /outbox/**; /send & /health stay public. Secret lives only in the gateway.

🚚 Why each technology

TECH	USED FOR	WHY
Redis	blocklist (hot path)	in-memory, O(1) Set check
Kafka	async event bus	durable, replayable, decouples
Mongo	event history	flexible docs, write-heavy
H2	outbox table	relational + transactional
Spring	sender	first-class Kafka/Redis
Go	store	goroutines: consumer + HTTP

Each store is chosen for its **data shape and access pattern** — fast lookups in Redis, a transactional state machine in a relational DB, a durable bus in Kafka, a flexible append-log in Mongo.

Known Limitations & Roadmap

 Open items — all additive, no re-architecture

LIMITATION	PLANNED RESOLUTION
Mock idempotency map in-memory	real vendor enforces it server-side
No client-level idempotency	client-supplied idempotency key
Single partition & replica	3 replicas, N partitions, ISR=2
No producer idempotence	<code>enable.idempotence</code> + <code>acks=all</code>
Unbounded history query	pagination + index on phoneNumber
Shared admin key	per-seller keys + tenant isolation
No outbox retention	scheduled archival job

 The thread

Start simple, then systematically stress-test: *what if Kafka dies? what if the worker crashes mid-step? what if a message is redelivered?* Each question exposed a real failure mode and led to a concrete fix.

 Where it stands

The outbox foundation, the end-to-end idempotency chain, and the public/admin split are **production-shaped**. The open items are scaling and multi-tenancy — the next layer, not dead-ends.

Thank you · Krishna Thakur · Distributed SMS Service · Questions welcome